

# Arena of Ratings: Real-Time Matchmaking Engine

## Final Exam Programming Question (30 Marks)

Language: C++

---

### Problem Story (Game Scenario)

You are building the matchmaking and leaderboard engine for a competitive game called **Arena of Ratings**. Each player is identified by a **unique integer rating**. Players may join or leave at any time, and the system must quickly answer queries such as:

- finding a player's information by rating,
- listing players in a rating interval in sorted order,
- finding the nearest opponent to a challenger rating,
- finding the next higher / previous lower rated player,
- computing rank and k-th player by rating,
- structural diagnostics such as height and leaf count,
- computing a duel path distance between two players inside the maintained index structure.

**Your task:** process commands and print outputs exactly as specified.

### Key Rule (Ordering)

The system is ordered **only by rating** (unique integer). All ordering-related operations (sorted range listing, neighbor queries, rank, k-th, etc.) are based exclusively on **rating**.

### Player Record

Each player record contains:

| Field                | Description                                                            |
|----------------------|------------------------------------------------------------------------|
| <b>rating (int)</b>  | Unique identifier used for ordering and search. No duplicates allowed. |
| <b>name (string)</b> | Player name (no spaces; underscores _ allowed).                        |
| <b>hp (int)</b>      | Health points ( $0 \leq hp \leq 10^9$ ).                               |

**Important:** Even though **hp** can change, the ordering is **always** by **rating** only.

### Input Format

- First line: integer  $Q$  = number of commands.
- Next  $Q$  lines: each line is one command (formats given below).

## Constraints

- $1 \leq Q \leq 200000$
- **rating** fits in 32-bit signed integer range.
- **name** has no spaces (underscores allowed).
- Your program must be efficient for large input sizes.

## Output Rules

Print output **only** for commands that require output, and match the specified text **exactly** (case-sensitive).

## Commands (Operations) — Full Specification

### 1) JOIN

Add a new player into the arena.

**Command:**

JOIN rating name hp

**Behavior:**

- If a player with the same **rating** already exists, do not change anything and print:

DUPLICATE

- Otherwise insert the player and print:

JOINED

### 2) LEAVE

Remove a player permanently (delete the record).

**Command:**

LEAVE rating

**Behavior:**

- If the player does not exist, print:

NOT\_FOUND

- Otherwise delete the player and print:

LEFT

**Deletion requirement (high marks):** your deletion must correctly handle all cases:

- deleting a node with no children,
- deleting a node with one child,
- deleting a node with two children.

If a deleted node has two children, you must preserve correct ordering so future searches and ordered queries remain correct.

### 3) STATUS

Query a player by rating.

**Command:**

STATUS rating

**Behavior:**

- If not found, print:

NOT\_FOUND

- If found, print exactly one line:

rating name hp

### 4) DAMAGE

Decrease a player's health.

**Command:**

DAMAGE rating amount

**Behavior:**

- If the player does not exist, print:

NOT\_FOUND

- Otherwise update:

$hp \leftarrow \max(0, hp - \text{amount})$

and print:

DAMAGED hp

where hp is the updated value.

### 5) HEAL

Increase a player's health.

**Command:**

HEAL rating amount

**Behavior:**

- If the player does not exist, print:

NOT\_FOUND

- Otherwise update:

$$hp \leftarrow hp + \text{amount}$$

and print:

HEALED hp

where hp is the updated value.

**6) MATCH**

Find the best opponent for a new challenger rating  $X$ .

**Command:**

MATCH  $X$

**Goal:** Choose the existing player whose rating is **closest** to  $X$ .

Define:

- $P$  = the player with the **largest rating strictly less than  $X$** .
- $S$  = the player with the **smallest rating strictly greater than  $X$** .

**Selection rules:**

- If the arena has no players at all, print: EMPTY
- If only  $P$  exists, choose  $P$ .
- If only  $S$  exists, choose  $S$ .
- If both exist:
  - Compare distances:  $|X - P.\text{rating}|$  and  $|S.\text{rating} - X|$ .
  - Choose the smaller distance.
  - If tied, choose the **lower rating** (choose  $P$ ).

**Output:**

- If an opponent is chosen, print:

rating name hp

## 7) NEXT

Find the player with the smallest rating strictly greater than  $X$ .

**Command:**NEXT  $X$ **Behavior:**

- If no such player exists, print: NONE
- Otherwise print: `rating name hp`

## 8) PREV

Find the player with the largest rating strictly smaller than  $X$ .

**Command:**PREV  $X$ **Behavior:**

- If no such player exists, print: NONE
- Otherwise print: `rating name hp`

## 9) RANGE

Print all players whose ratings lie within  $[L, R]$  inclusive, in increasing order of rating.

**Command:**RANGE  $L$   $R$ **Behavior:**

- If there are no players with  $L \leq \text{rating} \leq R$ , print: EMPTY
- Otherwise print each matching player on a new line in strictly increasing `rating` order:

`rating name hp`

## 10) RANK

Count how many players have rating strictly less than  $X$ .

**Command:**RANK  $X$ **Output:** print one integer on one line:`count`**Definition:** `count` = number of players with `rating`  $< X$ .

## 11) KTH

Print the  $k$ -th smallest player by rating (1-indexed).

**Command:**

KTH  $k$

**Behavior:**

- If  $k < 1$  or  $k > (\text{number of players})$ , print: `OUT_OF_RANGE`
- Otherwise print the player who has the  $k$ -th smallest rating:

rating name hp

## 12) DUEL

Compute the number of edges on the unique path between two ratings in the maintained index structure.

**Command:**

DUEL  $A$   $B$

**Behavior:**

- If either rating  $A$  or  $B$  does not exist, print: `NOT_FOUND`
- Otherwise print:

DIST  $d$

where  $d$  is the number of edges on the unique path between the two nodes inside your stored structure.

**Notes:**

- $d = 0$  if  $A$  and  $B$  are the same node.
- $d = 1$  if one node is a direct parent/child of the other.
- The answer depends on the current shape formed by insertions/deletions.

## 13) STATS

Print game + structural statistics.

**Command:**

STATS

**Output exactly 5 lines:**

```
PLAYERS <n>
MIN <min_rating_or_NONE>
MAX <max_rating_or_NONE>
HEIGHT <h>
```

```
LEAVES <leaf_count>
```

**Definitions:**

- **n** = total players currently stored.
- **MIN** = smallest rating among all players, or **NONE** if empty.
- **MAX** = largest rating among all players, or **NONE** if empty.
- **LEAVES** = number of nodes with no children.
- **HEIGHT** (structure height):
  - empty structure height =  $-1$
  - single node height =  $0$
  - otherwise  $1 + \max(\text{height}(\text{left}), \text{height}(\text{right}))$

**Implementation Restrictions (Strict)**

- You must implement your own pointer-based dynamic structure.
- **Do not use:** `std::map`, `std::set`, `multimap`, `multiset`, or any ordered container that directly provides neighbor queries/rank/k-th.
- Use fast I/O if needed (`ios::sync_with_stdio(false)`; and `cin.tie(nullptr)`).

**Marking Breakdown (30 Marks)**

| Component                                        | Marks |
|--------------------------------------------------|-------|
| Correct parsing + output formatting + edge cases | 4     |
| JOIN, STATUS, duplicate handling                 | 5     |
| DAMAGE, HEAL updates                             | 3     |
| NEXT, PREV, MATCH (tie rules)                    | 6     |
| RANGE sorted output correctness                  | 4     |
| LEAVE deletion correctness (0/1/2-child cases)   | 6     |
| RANK, KTH correctness after updates              | 1     |
| DUEL distance correctness                        | 1     |
| STATS correctness (min/max/height/leaves)        | 0     |

*(Instructor note: you may adjust mark distribution as needed.)*

**Sample Test Case 1****Input**

```
16
JOIN 50 Ares 120
JOIN 20 Luna 100
```

```
JOIN 70 Neo 90
JOIN 10 Ivy 80
JOIN 30 Omar 110
MATCH 25
NEXT 30
PREV 30
RANGE 15 55
KTH 3
RANK 50
DUEL 10 70
LEAVE 20
RANGE 0 100
STATS
```

## Output

```
JOINED
JOINED
JOINED
JOINED
JOINED
20 Luna 100
50 Ares 120
20 Luna 100
20 Luna 100
30 Omar 110
50 Ares 120
30 Omar 110
3
DIST 3
LEFT
10 Ivy 80
30 Omar 110
50 Ares 120
70 Neo 90
PLAYERS 4
MIN 10
MAX 70
HEIGHT 2
LEAVES 2
```

## Sample Test Case 2 (Boundary + Deletion Stress)

### Input

```
14
JOIN 40 Zed 150
JOIN 20 Amy 90
JOIN 60 Bo 110
JOIN 50 Cy 70
JOIN 80 Di 200
LEAVE 60
NEXT 80
PREV 20
KTH 4
```

```
RANK 50
DUEL 20 50
RANGE 45 90
STATS
MATCH 100
```

## Output

```
JOINED
JOINED
JOINED
JOINED
JOINED
LEFT
NONE
NONE
80 Di 200
2
DIST 3
50 Cy 70
80 Di 200
PLAYERS 4
MIN 20
MAX 80
HEIGHT 2
LEAVES 2
80 Di 200
```

**End of Question.**